Модульное дерево поведения

Проект 8

Оглавление

Основная концепция.....1
Узел.....1
Узел действия.....1
Узел составного действия.....1
Последовательность.....
....2 Селектор.....2
Параллельно.....3
Узел декоратора.....3

Цикл.....3 Ограничение количества одновременных пользователей.....4
Предоставленные узлы.....4
Дерево.....5
Язык разметки (XML).....5

Пример: Monster.....5
Реализация на C++.....6
Модель памяти.....6
Данные конфигурации.....6 Данные времени выполнения.....7 Интеллектуальные указатели.....7 Реализация нового узла.....7

Создание узла.....7 Открытие доступа к вашему узлу.....9 Отчеты об ошибках в вашем узле.....9 Именование вашего узла.....9
Переменные.....9 Сценарии Lua.....10 Временные метки.....11
События.....11
Отладка и дерево Визуализация.....12

Агенты слэширования.....12
Добавление пользовательского текста отладки.....13 Ведение журнала и трассировка.....14 Точка останова.....14 Компиляция с отладочной информацией.....14 Проход через корень.....14 Передовая практика.....15
Именование узлов.....15
Именование временных меток.....15

Основная концепция

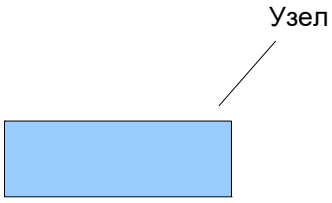
Модульное дерево поведения — это набор концепций, которые позволяют людям создавать поведение агентов, управляемых компьютером.

Вместо написания большого количества сложного кода на C++ или каком-либо другом общем языке программирования, модульное дерево поведения позволяет описывать поведение на высоком уровне.

уровне, не думая об указателях, памяти, компиляторах, классах и т. д.

Концепции и реализация ориентированы на быструю итерацию и повторное использование.

Узел



Узел — это самая фундаментальная концепция, которая вам нужна.

понимать. Это как кубик Лего. Вы строите поведение,

объединение множества различных узлов.

Все узлы имеют одинаковый интерфейс — он выполняет задачу, которая может либо завершиться успешно, либо потерпеть неудачу.

Однако классифицировать всегда полезно, поэтому я рассмотрю несколько распространенных шаблонов узлов.

Узел действия

Обычно мы говорим, что узел — это узел действия.

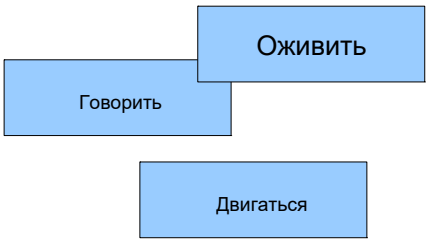
когда он выполняет какое-то действие. Для

например, у вас может быть один узел, который создает

агент воспроизводит анимацию, другую, которая делает это

говорить, и третья, которая заставляет его двигаться

где-то.



Узел действия ничем не отличается от любого другого узла, я не могу это подчеркнуть!

Термин «действие» используется просто для облегчения общения.

Составной узел

Представьте, что вы хотите создать монстра, который указывает на игрока, рычит, а затем преследует его.

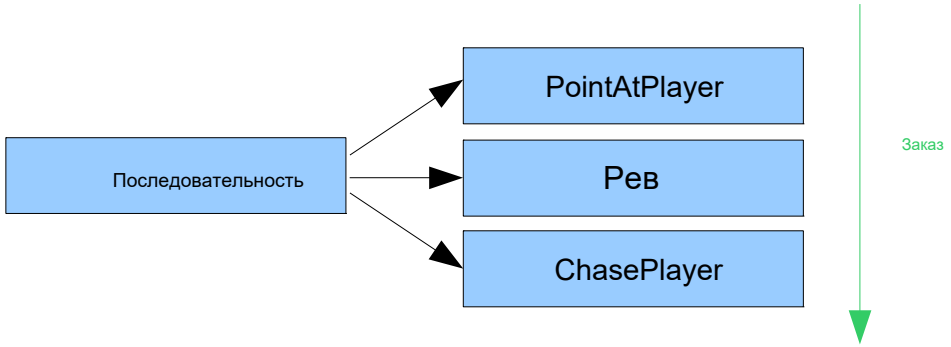
он или она. Вы захотите выполнить эти действия последовательно, и именно здесь

составные узлы пригодятся.

Они содержат набор узлов, которые выполняются в определенном порядке. Узлы в этом коллекции называются детьми.

Последовательность

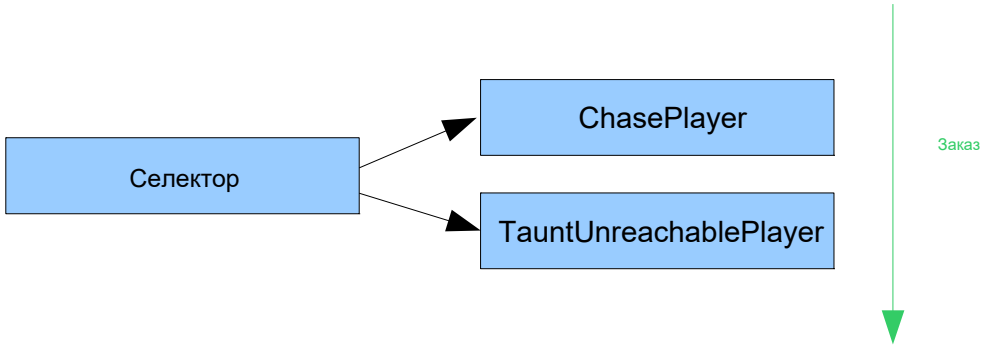
Например, узел Sequence последовательно выполняет свои дочерние элементы (свою коллекцию узлы) по порядку, как в нашем примере выше с точкой, ревом и погоней. Сначала это будет и если это удалось, мы начинаем реветь. Если и это удалось, начинаем преследование. Если все дети преуспели, мы считаем, что сама Последовательность увенчалась успехом.



Если, однако, какой-либо из детей потерпел неудачу, мы немедленно считаем, что сама Последовательность потерпели неудачу, не перейдя к следующему дочернему узлу.

Селектор

Давайте представим другой сценарий! На этот раз мы хотим, чтобы наш монстр преследовал игрока, но если он не может до него дотянуться, он должен кричать: «Приди и сразись со мной, трус!» Такое поведение может быть достигнуто с помощью составного узла Selector с двумя дочерними элементами (коллекция из двух узлов). Первый дочерний элемент — узел ChasePlayer, а второй дочерний элемент — узел. TauntUnreachablePlayer узел. Селектор сначала выполнит ChasePlayer, что завершится неудачно. потому что нет возможности подняться в здание. Затем Селектор переходит к следующему node в своей коллекции, которая в нашем случае — TauntUnreachablePlayer. Это получается просто хорошо, и монстр насмехается над игроком.



Селектор завершится успешно, как только дочерний узел завершится успешно.

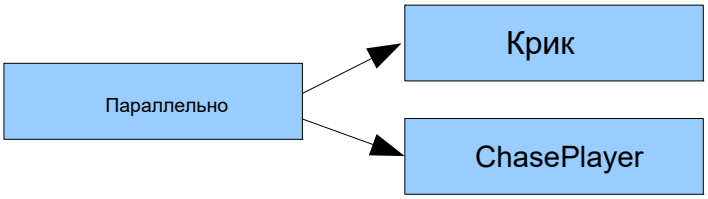
Селектор завершится неудачно, если все дочерние узлы были опробованы и все они потерпели неудачу.

Он идеально подходит для опробования различных тактик в игре или создания запасных вариантов поведения в игре.

если что-то неожиданное пойдет не так.

Параллельно

Представьте, что нам нужен монстр, который будет кричать и преследовать игрока одновременно, а не один перед другим. Вы бы хотели, чтобы они оба происходили параллельно, и вы можете сделать просто это путем создания дочерних элементов Scream и ChasePlayer параллельного составного узла.



Узел-декоратор

Узел-декоратор — это узел, который добавляет какую-то функциональность к другому узлу.

не зная, как работает этот другой узел и что он делает.

Петля

Представьте, что вы хотите воспроизвести анимацию 5 раз подряд. Теперь ты можешь просто идти вперед и реализовать узел Animate, который воспроизводит анимацию 5 раз, прежде чем она будет успешной.
Это совершенно нормально. Но что, если позже вы поймете, что хотите кричать 4 раза подряд?
Еще раз: вы можете повторно реализовать ту же функциональность цикла в узле Scream.
также. Тем не менее, настоятельно рекомендуется следить за возможным повторным использованием.
и вместо этого реализовать узел-декоратор, который выполняет эту работу.

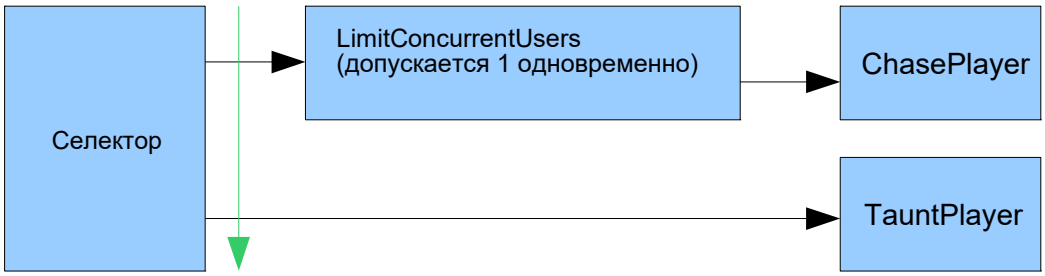
В нашем случае мы просто создадим узел-декоратор цикла с одним дочерним узлом. Мы устанавливаем узел Loop для выполнения цикла 5 раз, а затем мы добавляем узел Animate в качестве его дочернего узла. Когда узел Loop обновляется (или «отмечается галочкой»), он обновляет свой дочерний узел (в нашем случае Анимировать). Когда анимация закончит играть, он сообщит, что это удалось. Узел цикла отслеживает это и однажды снова отметьте узел Animate. После 5 раз узел Loop сообщит, что все удалось.



Ограничить количество одновременных пользователей

Узел декоратора LimitConcurrentUsers позволяет указать, сколько пользователей должно разрешено одновременно использовать узел.

Сценарий! Представьте, что у вас есть три монстра, и вы хотите, чтобы один из них преследовал игрок и остальные, чтобы насмехаться над игроком. Для этого мы могли бы создать такую структуру:



Первый монстр пройдет через селектор к узлу LimitConcurrentUsers.
Узел LimitConcurrentUsers сообщает, что в настоящее время никто не использует его дочерний узел, и давайте перейдем к ChasePlayer. LimitConcurrentUsers теперь помнит, что мы прошел и эффективно запирает дверь.

Второй монстр также перейдет через Селектор к LimitConcurrentUsers.
узел. Только на этот раз нам сказали, что здесь уже кто-то есть, и нас не пускают.
in. Узел LimitConcurrentUsers сообщает об ошибке. Если вы прочитали раздел «Селектор»
вы, возможно, помните, что теперь он попробует своего следующего дочернего элемента, в данном случае это TauntPlayer.
Монстр теперь насмехается.

Третий монстр получает ту же историю, что и второй – узел LimitConcurrentUsers.
говорит нам, что мы не можем пройти, потому что там уже кто-то есть.

Это очень мощно! Вы можете настроить количество пользователей, которым разрешено преследовать игрока.
одновременно, и вы можете сделать поведение ChasePlayer настолько сложным или простым, как вам бы хотелось. Опробовать идеи можно быстро и легко, и вам не придется так сильно беспокоиться о проектирование заранее.

Предоставленные узлы

Многие строительные блоки (конечные автоматы и т. д.) уже предусмотрены. Подробнее читайте на
http://confluence/display/C3/Modular+Behavior+Tree+Nodes+Documentation

Дерево

Структуры и коллекции узлов, которые мы видели в предыдущем разделе, относятся к в качестве деревьев поведения – потому что все они имеют корень и разветвляются все больше и больше в листья. В моих визуальных представлениях они выглядят как упавшие деревья, но вы получаете суть.

Язык разметки (XML)

Любой язык разметки подойдет, но мы решили описывать деревья с помощью XML, поскольку у нас уже была программа чтения XML, и мы использовали файлы XML для многих других систем, поэтому люди команду это устраивало.

Деревья поведения загружаются каждый раз, когда пользователь заходит в игру в редакторе.

Пример: Монстр

Файл: Скрипты/AI/BehaviorTrees/MonsterTree.xml

Один монстр за раз может преследовать игрока. Остальные монстры выстоят вокруг и насмехаться над игроком.

```
<BehaviorTree>
	<Корень>
		<Селектор>
			<LimitConcurrentUsers макс="1">
				<ChasePlayer />
			</LimitConcurrentUsers>
			<TauntPlayer />
		</Селектор>
	</Корень>
</BehaviorTree>
```

Реализация C++

Вы найдете весь код дерева модульного поведения, инкапсулированный в BehaviorTree.

namespace.

Модель памяти

Модульное дерево поведения занимает относительно небольшой объем памяти. Это достигается

разделяя неизменяемые данные (только для чтения) между экземплярами дерева и имея только

память, выделенная для вещей, необходимых в текущей ситуации.

Память разделена на две категории: данные конфигурации и данные времени выполнения.

Данные конфигурации

Позвольте мне объяснить, как загружается следующее дерево.

```
1 <Последовательность>
2 <Имя связи="Привет" />
3 <Animate name="LookAround" />
4 <Продолжительность ожидания="2.0" />
5 <Имя связи="WeShouldGetSomeFood" />
6 </Последовательность>
```

Когда вы загрузите приведенное выше дерево поведения из файла XML, вы получите поведение

Шаблон дерева, содержащий все данные конфигурации, которые вы видите выше. Будет

узел последовательности с четырьмя детьми; узел связи, узел анимации, узел ожидания,

и еще один узел связи.

Данными конфигурации будут название анимации, продолжительность и т. д. Эти данные не меняются –

всегда.

Память для данных конфигурации выделяется из кучи уровня и освобождается

выгрузка уровня при запуске игры через лаунчер или при выходе игрока

игровой режим и возвращается в режим редактирования в редакторе.

Данные времени выполнения

Когда вы создаете агента, использующего это дерево поведения, экземпляр дерева поведения будет создан и связан с агентом. Он указывает на template и содержит только несколько вещей, специфичных для экземпляра, таких как переменные и временные метки (которые мы увидим позже).

Когда дерево отмечено галочкой с учетом нашего агента, оно сначала посещает узел Последовательность. Базовая система увидит, что это происходит впервые, и выделит для этого объект данных времени выполнения. конкретная конфигурация узла для этого конкретного агента. Вы следили за этим? Каждый агент получит свой собственный объект данных времени выполнения при посещении узлов дерева. Данные времени выполнения объект сохраняется как long, пока агент посещает узел (это может быть несколько кадров), но будет быть освобожден, как только агент покинет узел.

Память для данных времени выполнения выделяется распределителем сегментов. Это чтобы минимизировать фрагментация, поскольку данные времени выполнения обычно занимают всего несколько байт и часто выделяются и освобождается, и тогда будет сложнее найти непрерывный блок памяти для больших фрагменты данных. Распределитель ковша очищается по уровню выгрузки.

Умные указатели

Интеллектуальные указатели Boost используются во всем коде модульного дерева поведения. Раньше это было уметь безопасно передавать данные и максимально избегать необработанных указателей. Управление памятью осуществляется ядром системы. В некоторых местах unique_ptr из C++11 подойдет идеально, но мы используем shared_ptr из boost для причины совместимости.

Реализация нового узла

Создание вашего узла

```
#include <BehaviorTree/Node.h>

класс MyNode : public BehaviorTree::Node {

    typedef BehaviorTree::Узел BaseClass;

public:

    // Каждый экземпляр узла в дереве агента будет иметь // объект данных времени выполнения. // // Эти данные сохраняются с момента посещения узла // до момента его выхода. // // Если этот struct опущен, код не скомпилируется.
```

```
// // Это будут переменные типа 'bestPostureID', 'shotsFired' и т. д. struct
RuntimeData { };



MyNode() : m_speed(0.0f) { }



// Здесь вы загружаете данные конфигурации из файла XML // в члены узла. В них можно
записать только на этапе загрузки // и они концептуально неизменяемы (только для чтения)
позже, во время работы игры. virtual LoadResult LoadFromXml(const XmlNodeRef& xml, const
LoadContext& контекст) {

    if (BaseClass::LoadFromXml(xml, context) == LoadFailure)
        вернуть LoadFailure;

    xml->getAttr("скорость", m_speed);

вернуть LoadSuccess; }


protected:

// Вызывается прямо перед первым обновлением virtual void
OnInitialize(const UpdateContext& context) {

    BaseClass::OnInitialize(контекст);

// Необязательно: доступ к данным времени выполнения, например
RuntimeData& runtimeData = GetRuntimeData<RuntimeData>(context); }


// Вызывается, когда узел завершается virtual void
OnTerminate(const UpdateContext& context) {

    BaseClass::OnTerminate(контекст);

// Необязательно: доступ к данным времени выполнения, например
RuntimeData& runtimeData = GetRuntimeData<RuntimeData>(context); }


Обновление статуса virtual(const UpdateContext& контекст) {

    // Выполните код обновления и сообщите, был ли ваш // узел
    успешным, неудачным или он работает, и ему требуется больше //
    времени для выполнения своей задачи.


    // Необязательно: доступ к данным времени выполнения, например
    RuntimeData& runtimeData = GetRuntimeData<RuntimeData>(context);

вернуть успех; }


// Обработка любых входящих событий, отправленных на этот узел virtual void
HandleEvent(const EventContext& context, const Event& event) {

// Необязательно: доступ к данным времени выполнения, например
RuntimeData& runtimeData = GetRuntimeData<RuntimeData>(context); }


private:

// Сохраните здесь все данные конфигурации узла. // Это будут
неизменяемые вещи, такие как 'maxSpeed', 'duration', // 'threshold',
'impulsePower', 'soundName' и т. д. float m_speed; };



// Генерируем объект, специализирующийся на создании узла вашего типа // по запросу
фабрики узлов. Макрос помещает сюда глобальную переменную.
GenerateBehaviorTreeNodeCreator(MyNode);
```

Раскрытие вашего узла

Чтобы иметь возможность использовать новый узел, вам необходимо предоставить его фабрике узлов.

```
BehaviorTree::INodeFactory& Factory = gEnv->pAISystem->GetIBehaviorTreeManager()->GetNodeFactory();
ExposeBehaviorTreeNodeToFactory(фабрика, MyNode);
```

Отчеты об ошибках на вашем узле

Используйте класс ErrorReporter для сообщения об ошибках и предупреждениях на вашем узле. Это позволит вам войти сообщение в формате printf и автоматически включать всю доступную информацию о ваш узел, например номер строки XML, имя дерева, тип узла и т. д.

```
ErrorReporter(*this, context).LogError("Не удалось скомпилировать код Lua '%s'", code.c_str());
```

Именование вашего узла

См. главу «Лучшие практики».

Переменные

В идеале мы бы реализовали переменную/регистр/доску общего назначения. системе, но поскольку мы были short вовремя, мы повторно использовали код переменной выбора из Вместо этого дерево выбора поведения (BST). Мы можем статически объявлять переменные в XML и описать, как они изменятся, когда мы получим сигналы AI (отдельная концепция, но в основном это просто именованные текстовые сообщения в системе AI).

Пример агента, который движется к цели, если она видима, в противном случае анимируется:

```
<BehaviorTree>
	<Переменные>
		<Имя переменной="TargetVisible" />
	</Переменные>
	<SignalVariables>
		<Signal name="OnEnemySeen"variable="TargetVisible" value="true" />
		<Signal name="OnLostSightOfTarget"variable="TargetVisible" value="false" />
	</SignalVariables>
	<Корень>
		<Селектор>
			<IfCondition условие="TargetVisible">
				<Переместить к="Цель" />
			</IfCondition>
			<Animate name="LookAroundForTarget" />
		</Селектор>
	</Корень>
</BehaviorTree>
```

Lua-скрипты

Вы можете выполнить код Lua, встроенный в дерево, либо для запуска кода «выстрелил и забыл», либо для контролировать поток в дереве. Он идеально подходит для прототипирования или расширения функциональности. без необходимости создания новых узлов.

 ExecuteLua запускает фрагмент кода Lua. Это всегда удается.

```
<ExecuteLua code="DoSomething()" />
```

 LuaWrapper запускает один фрагмент кода Lua перед запуском дочернего элемента, а затем запускает другой бит кода Lua после того, как дочерний элемент добился успеха или потерпел неудачу.

```
<LuaWrapper onEnter=”StartParticleEffect()” onExit=”StopParticleEffect()”>
        <Move to="Обложка" />
</LuaWrapper>
```

 LuaGate запускает фрагмент кода Lua и запускает дочерний элемент, если код возвращает true. Это затем возвращает статус дочернего узла. Если код Lua возвращает false или не может выполнить, LuaGate вернет ошибку.

```
<LuaGate code="return IsAppleGreen()">
        <EatApple />
</LuaGate>
```

 AssertLua позволяет вам заявить о себе. Если это утверждение верно, узел завершается успешно, и если утверждение ложно, узел терпит неудачу.

```
<Последовательность>
        <AssertLua code="returnentent.someCounter == 75" />
        <AssertCondition условие=”TargetVisible” />
        <Переместить к=”Цель” />
</Последовательность>
```

У вас будет доступ к переменной «entity» во всех узлах Lua.

Код компилируется один раз при загрузке уровня в чистой игре, чтобы уменьшить фрагментацию, и только компилируется код для деревьев поведения, которые будут использоваться в какой-то момент уровня.

Временные метки

Временная метка — это информация о том, когда что-то произошло. Например, это полезно если вы хотите узнать, когда в персонажа в последний раз стреляли или попадали, или, может быть, когда он в последний раз мог видеть игрока. Или когда он в последний раз переезжал в новое место укрытия.

```
<BehaviorTree>
        <Временные метки>
                <Timestamp name="TargetSpotted" setOnEvent="OnEnemySeen" />
                <Timestamp name="ReceivedDamage" setOnEvent="OnEnemyDamage" />
                <Timestamp name="GroupMemberDied" setOnEvent="GroupMemberDied" />
        </Временные метки>
        <Корень>
                <Последовательность>
                        <WaitUntilTime c =”ReceivedDamage” isMoreThan=”5” orNeverBeenSet=”1” />
                        <Селектор>
                                <IfTime поскольку="GroupMemberDied" isLessThan=”10”>
                                        <MoveCautiouslyTowardsTarget />
                                </IfTime>
                                <MoveConfidentallyTowardsTarget />
                        </Селектор>
                </Последовательность>
        </Корень>
</BehaviorTree>
```

Необязательно: временные метки могут быть объявлены как эксклюзивные друг для друга. Например, TargetSpotted и TargetLost могут быть эксклюзивными друг для друга, поэтому предыдущий очищается. Вы не можете видеть игрока и в то же время считать его проигравшим.

События

В системе AI есть так называемые сигналы AI. По сути, они называются текстом. сообщения, отправленные агентам. Примеры: OnBulletRain, OnEnemySeen и т. д. Нам хотелось, чтобы узлы в нашем дереве, чтобы иметь возможность их слушать и реагировать на них.

Мы не хотели, чтобы модульное дерево поведения было слишком тесно связано с сигналами AI. Итак, у нас есть внутренняя концепция, называемая событием модульного дерева поведения.

Сигналы AI принимаются, преобразуются в события модульного дерева поведения и отправляются. к корневому узлу, который передает их вниз по работающим узлам дерева.

```
<Последовательность>
        <WaitForEvent name="OnEnemySeen" />
        <Имя связи="ThereHeIs" />
</Последовательность>
```

Отладка и визуализация дерева

Рубящие агенты

Установите для ai_DebugDraw значение 0 или 1 (по умолчанию — -1).

Посмотрите на агента и нажмите цифровую клавиатуру '/' («косая черта»), и дерево агента будет выглядеть так:

визуализировано. Существуют и другие способы выбора агента для начала отладки:

• Позвоните «ai_DebugAgentближайший», чтобы выбрать ближайшего к камере агента.

• Вызовите «ai_DebugAgent centerview», чтобы выбрать агента в центре поля зрения камеры (аналогично косой черте).

• Позвоните «ai_DebugAgent MrPickles», чтобы выбрать агента по имени MrPickles.

• Вызовите «ai_DebugAgent» без каких-либо параметров, чтобы сбросить отлаженный агент.

Дерево имеет цветовую маркировку:

• Белый для узлов с пользовательской информацией (подробнее об этом в следующей главе).

• Синий для конечных узлов (они часто имеют особое значение при отладке).

• Остальное тускло-серого цвета.



Имя агента указано в верхней части экрана, а над ним — маленькая зеленая точка. персонаж в мире. Числа слева — это номера строк в файле XML.

Добавление пользовательского текста отладки

Визуализатор дерева поддерживает пользовательскую информацию об узлах. Это позволяет нам получить больше информации

просмотр глубины текущих частей дерева.

Например, вы можете увидеть название события, которого ожидает узел WaitForEvent, или

сколько времени будет продолжаться Timeout, прежде чем истечет время ожидания.

Переопределите GetDebugTextForVisualizer, чтобы добиться этого на своем узле.

```
#ifdef STORE_INFORMATION_FOR_BEHAVIOR_TREE_VISUALIZER
virtual void GetDebugTextForVisualizer(
        const UpdateContext и updateContext,
        stack_string и debugText) const
{
        debugText.Format("Скорость %f", m_speed);
}
#endif
```



Ведение журнала и отслеживание

Вы можете быстро диагностировать, что происходит, отслеживая сообщения журнала. Вывод такой:

<Журнал сообщения = «В меня стреляли, я убегаю» />

Ведение журнала настолько распространено, что для него существует встроенная поддержка:

```
<Последовательность>
        <QueryTPS name="CoverFromTarget" _startLog="Найти укрытие" _failureLog="Не удалось найти укрытие" />
        <Move to="Обложка" _startLog="Продвижение" _failureLog="Не удалось перейти" _successLog="Дополнительно" />
</Последовательность>
```

Зарезервированные атрибуты _startLog, _successLog и _failureLog считываются автоматически.

Сообщения журнала будут маршрутизироваться через объект, производный от

Интерфейс BehaviorTree::ILogRouter, поэтому вы сами решаете, где окажется информация. Для

Crysis 3 мы направляем его в личный журнал, который представляет собой short историю сообщений журнала для каждого

агент, который отображается на экране для текущего «вырезанного» агента.

Сообщения журнала также записываются в записывающее устройство AI, поэтому вы можете перемещаться вперед и назад во времени.

наблюдать за порядком событий и что произошло.

Точка останова

Если вы хотите разбить дерево поведения в Visual Studio, вы можете добавить узел Breakpoint в дерево. Когда он попадет, вы получите контроль над Visual Studio и сможете шагнуть вперед.

Компилировать с отладочной информацией

DEBUG_MODULAR_BEHAVIOR_TREE необходимо определить. Crysis 3 делает это:

```
#if !define(_RELEASE) && (определено(WIN32) || определено(WIN64))
# определить DEBUG_MODULAR_BEHAVIOR_TREE
#endif
```

Падение сквозь корень

Если ваше дерево потерпит неудачу или преуспеет на всем пути до корневого узла, вы получите уведомление в

окно консоли и список недавно посещенных узлов и номера их строк будут

представлено.

[Ошибка] Модульное дерево поведения: корневой узел для сущности «HumanSoldier» FAILED.

Перезагрузка дерева в следующем кадре. (124)Двигай. (122) Селектор. (121) Последовательность.

Как сказано выше, дерево будет перезагружено в следующем кадре. Однако мы рассматриваем это как знак

что вы разработали свое дерево поведения не для того, чтобы справиться с только что произошедшей ситуацией.

Передовая практика

Рассматривайте эту информацию как предложения, основанные на том, что, по вашему мнению, делает вещи приятными и полезными.

легко читать. Это увеличивает скорость общения команды, если именование легко подобрать.

понимать. Все люди разные, и это может не относиться к вашему случаю.

Именование узлов

Назовите узлы действий так, чтобы они звучали как слово или предложение, которое вы используете для командования.

это для выполнения задачи за вас.

Хороший

□ Зацикливание □
Анимация □
LimitConcurrentUsers □
ExecuteLua □ Съемка □
AdjustCoverStance

Плохой

□ Быстро □
PathPredictor □
Банан □ Сценарий □
ActivationProcess

Именование временных меток

Кажется, хорошо давать названия временным меткам, чтобы они звучали как нечто

произошло, учитывая, что все они произошли в прошлом:

• TargetSpotted •
ReceivedDamage •
GroupMemberDied